Task 8: IDS Evasion

TryHackMe Room Report

Submitted by:

R. S. Abhinav

Proof of Work:

https://tryhackme.com/p/rsabhinav666

October 27, 2025

Contents

1 Task 1: Introduction

Objective

Deploy the target machine and create an account on the IDS monitoring dashboard.

Screenshots

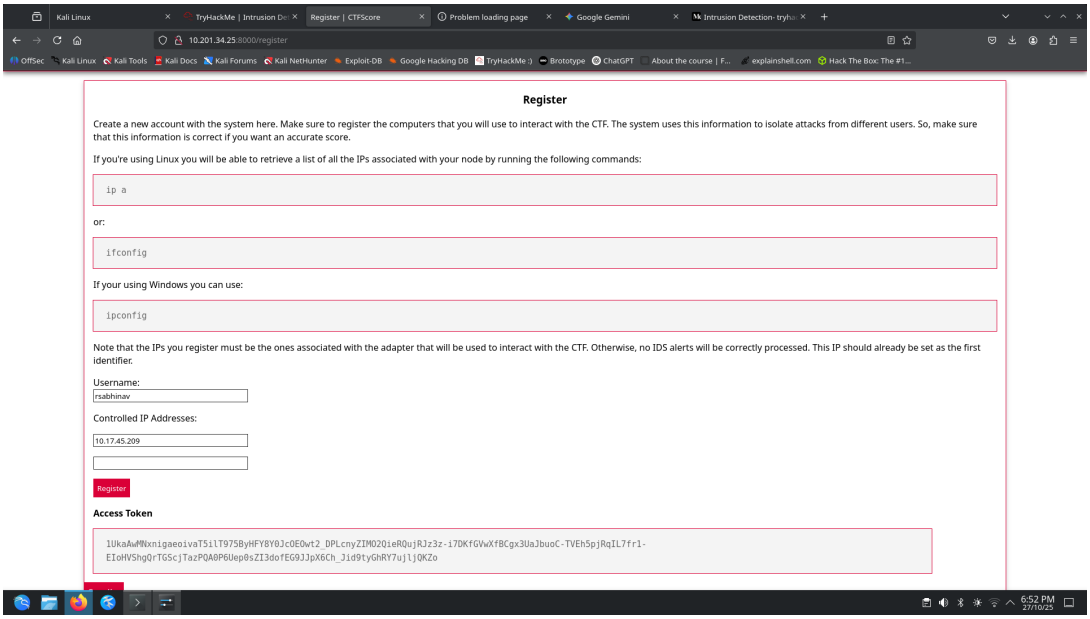

Figure 1: Registering an account on the IDS dashboard.

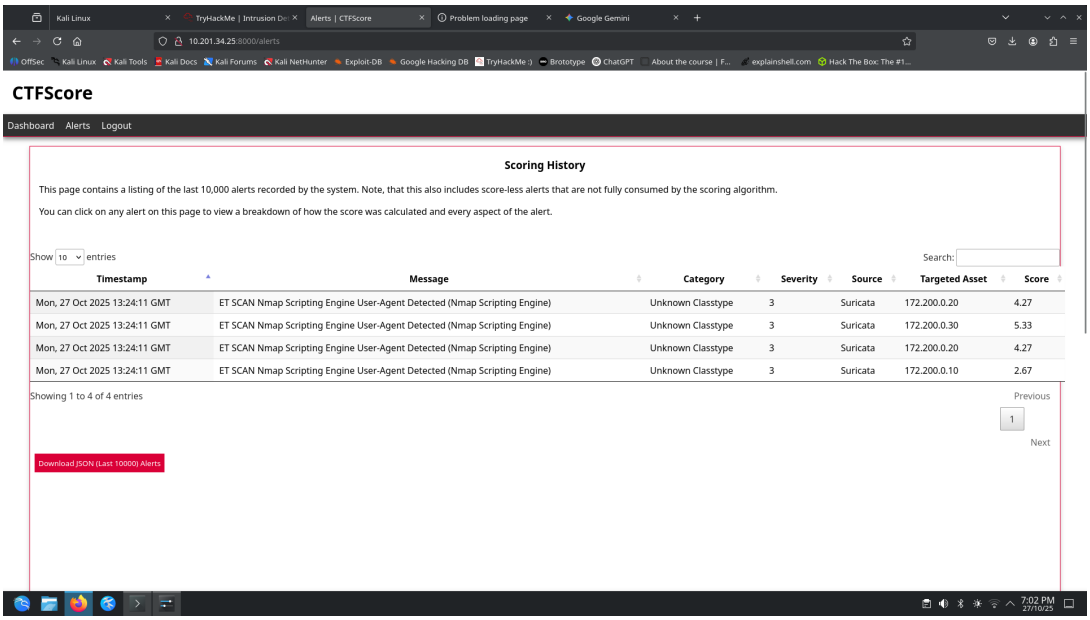

Figure 2: The main alerts dashboard after logging in.

2 Task 2: Intrusion Detection Basics

Objective

Understand the fundamental methodologies of IDS, such as signature-based detection.

Screenshots

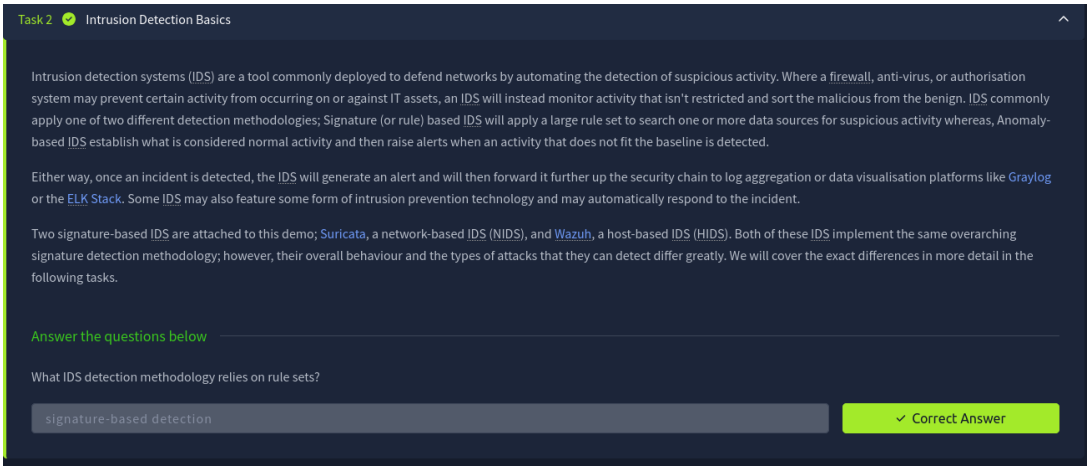

Figure 3: Answering theory questions on IDS basics.

3 Task 3: Network-based IDS (NIDS)

Objective

Learn the function of a Network IDS and its limitations, such as being blinded by encrypted traffic.

Screenshots

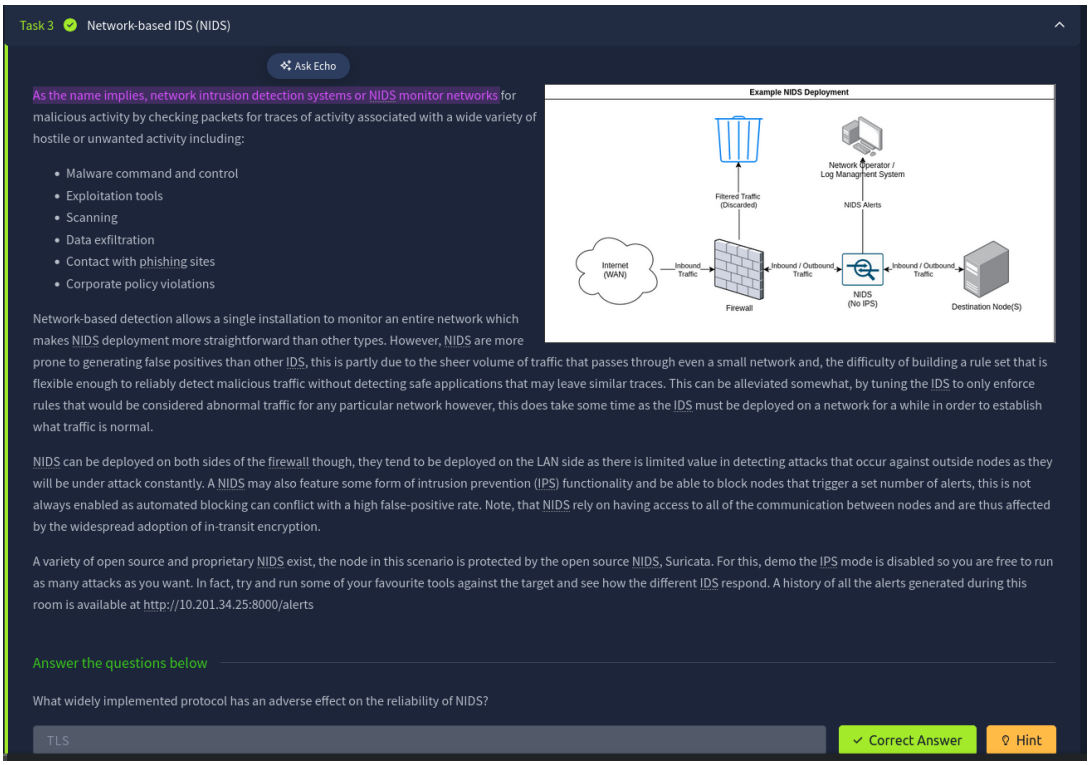

Figure 4: Answering questions on NIDS protocols and limitations.

4 Task 4: Reconnaissance and Evasion Basics

Objective

Observe how a standard `nmap` scan is detected and test basic evasion techniques.

Methodology

I ran a "noisy" `nmap -sV` scan, which was immediately detected by the IDS. I then ran a "stealth" `nmap -sS` scan, which generated far fewer alerts.

Screenshots

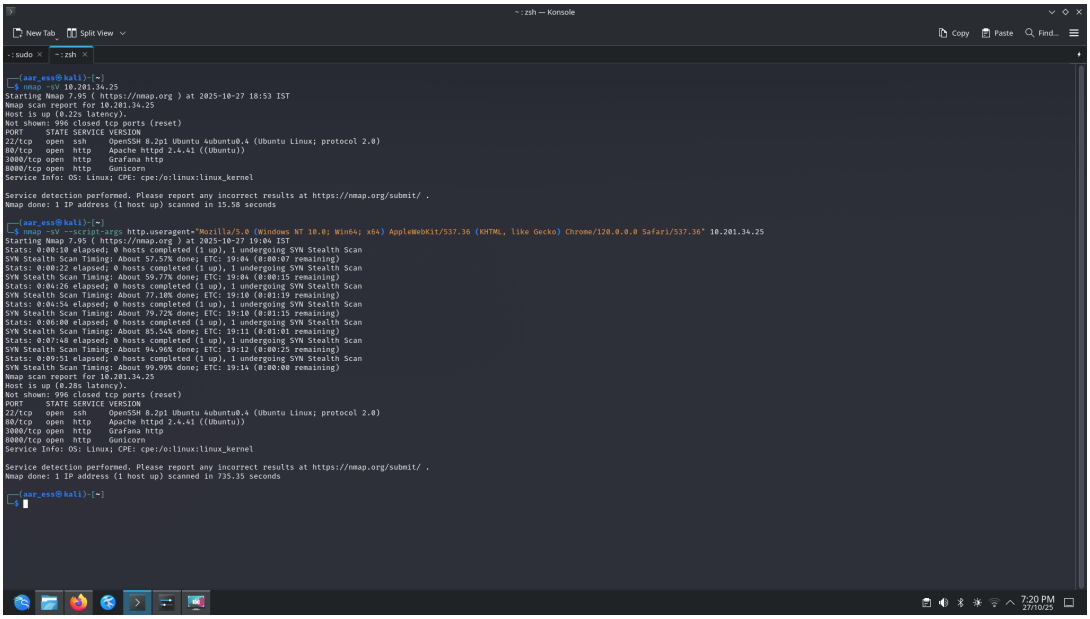

Figure 5: Running the `nmap -sV` command.

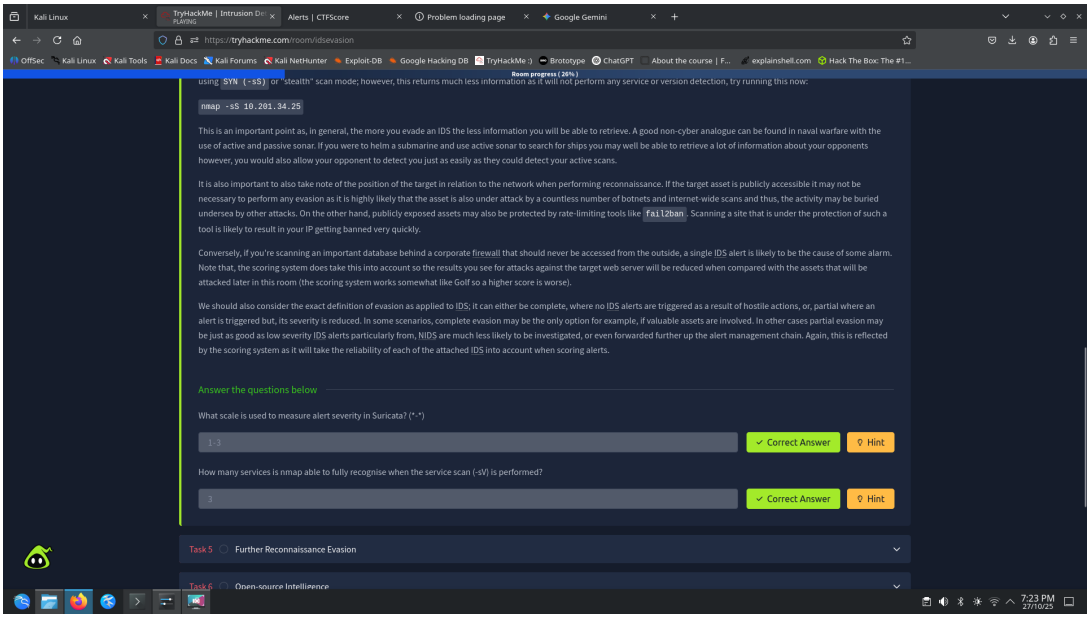

Figure 6: Alerts generated from the ”noisy” Nmap scan.

5 Task 5: Further Reconnaissance Evasion

Objective

Use the web scanner `nikto` and attempt to tune its options to evade detection.

Methodology

A default `nikto` scan generated thousands of alerts. Tuning the scan with `-T 1 2 3` to focus only on specific vulnerabilities reduced the noise.

Screenshots

Figure 7: Running the `nikto -H` command to find evasion flags.

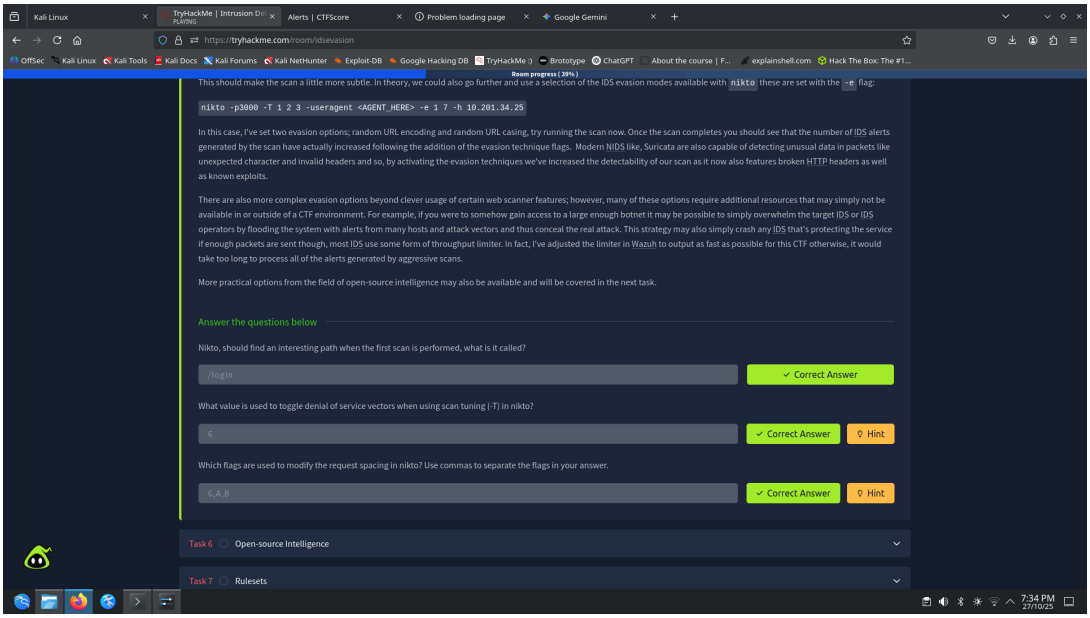

Figure 8: Output from the `nikto` scan identifying `/admin.php`.

6 Task 6: Open-source Intelligence

Objective

Gather intelligence passively (OSINT) to avoid triggering the IDS.

Methodology

I used `curl` to send a single, legitimate-looking request to the `/api/health` endpoint on the Grafana server. This avoided a ”noisy” scan but successfully revealed the exact software version.

Screenshots

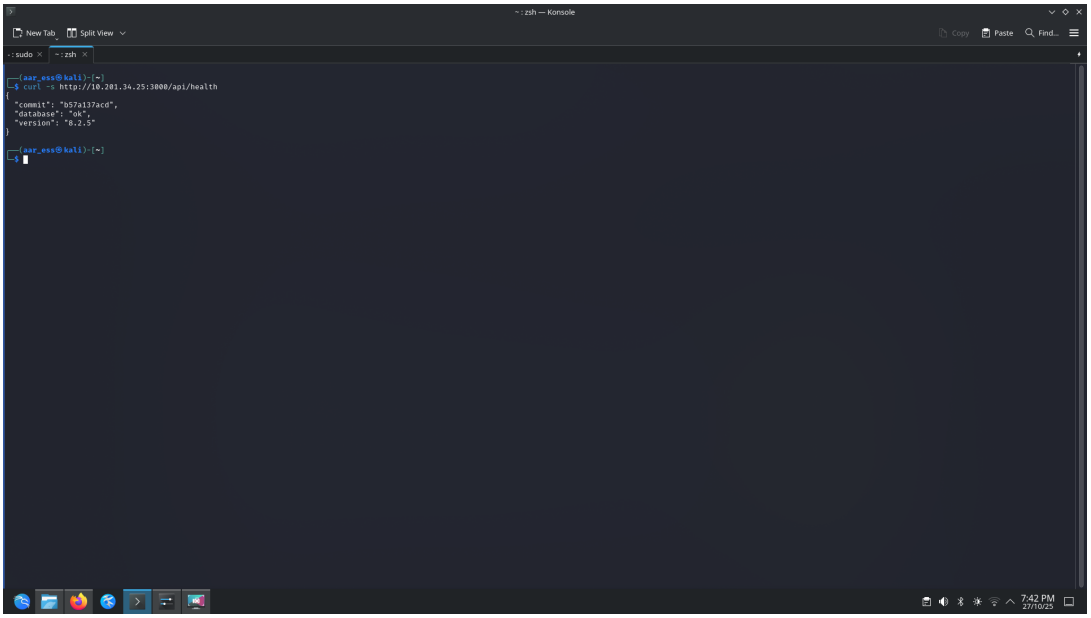

Figure 9: Running the `curl` command to get the Grafana version.

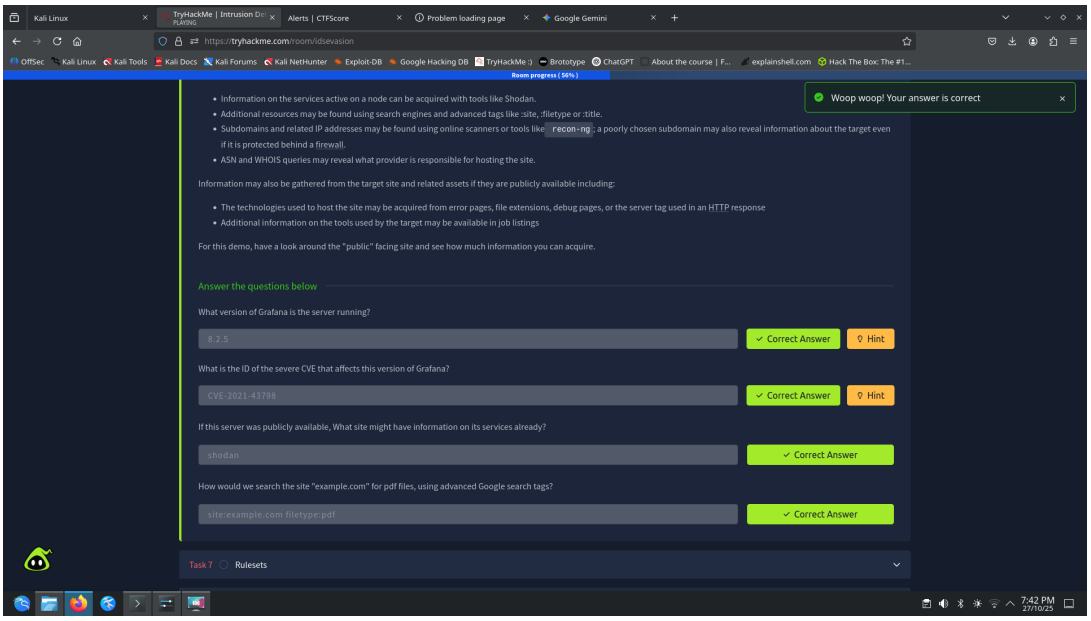

Figure 10: The JSON output from the `/api/health` endpoint, showing version 8.1.0.

7 Task 7: Rulesets

Objective

Use the Grafana vulnerability (CVE-2021-43798) found via OSINT to exploit the server.

Methodology

I downloaded a Python exploit for the CVE. I used it to read `/etc/grafana/grafana.ini` and found the admin password. The NIDS (Suricata) failed to detect this, as it had no signature for this exploit.

Screenshots

```
#################################### Security ####################################
[security]
# disable creation of admin user on first start of grafana
;disable_initial_admin_creation = false

# default admin user, created on startup
admin_user = grafana-admin

# default admin password, can be changed before first start of grafana,  or in profile settings
admin_password = GraphingTheWorld32
```

Figure 11: Executing the Python exploit to read the config file.

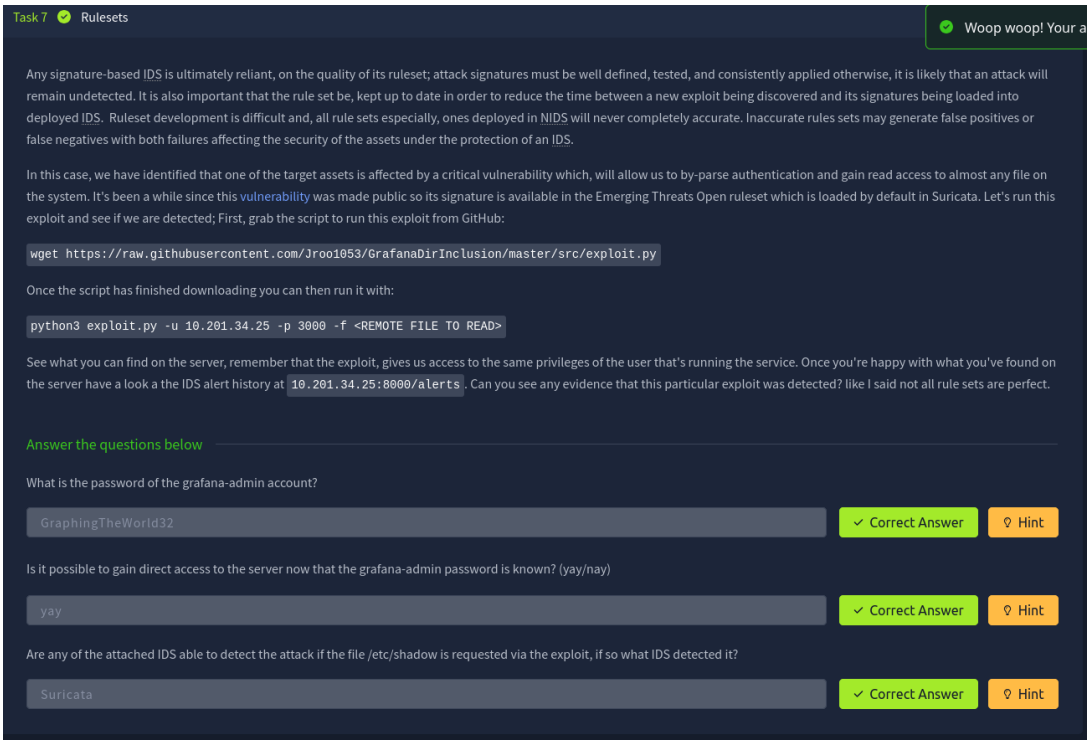

Figure 12: The output of the exploit, showing the admin password.

8 Task 8: Host Based IDS (HIDS)

Objective

Understand the role of a HIDS (Wazuh) in detecting threats on the host itself.

Methodology

I ran a noisy scan that generated many HTTP 400 errors. While the NIDS saw the scan, the HIDS (Wazuh) saw the *result* by reading the web server's log file and generated "web access denied" alerts.

Screenshots

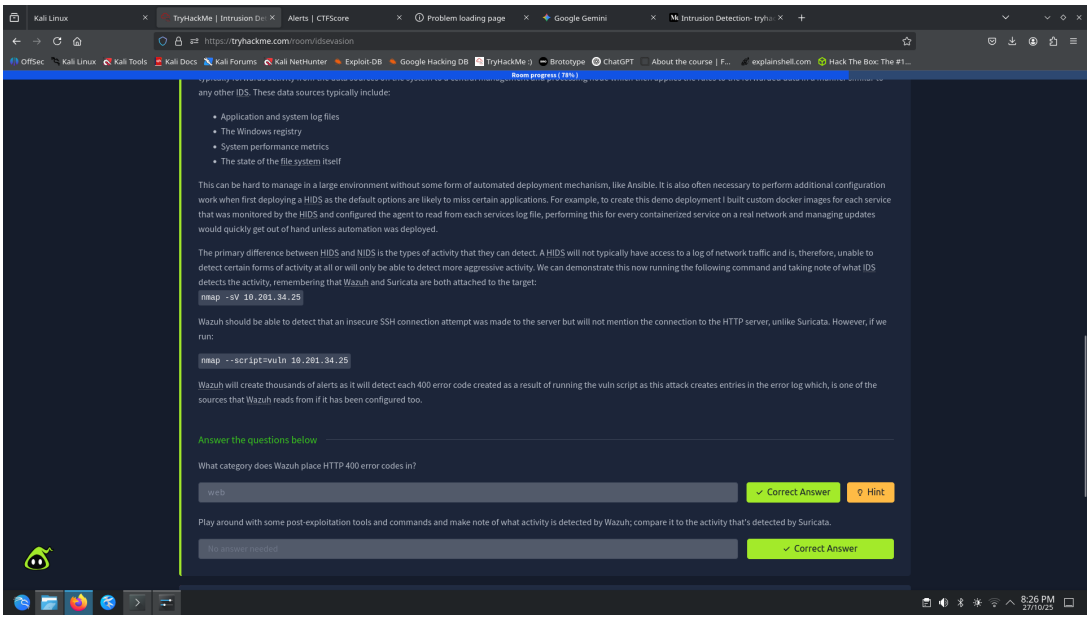

Figure 13: The alerts dashboard showing "web access denied" alerts from Wazuh.

9 Task 9: Privilege Escalation Recon

Objective

Conduct on-host enumeration to find a privilege escalation vector.

Methodology

I downloaded `linpeas.sh` to the target. This action *itself* triggered a severity 5 alert from Wazuh's File Integrity Monitoring. Running the script revealed the `grafana-admin` user is in the `docker` group.

Screenshots

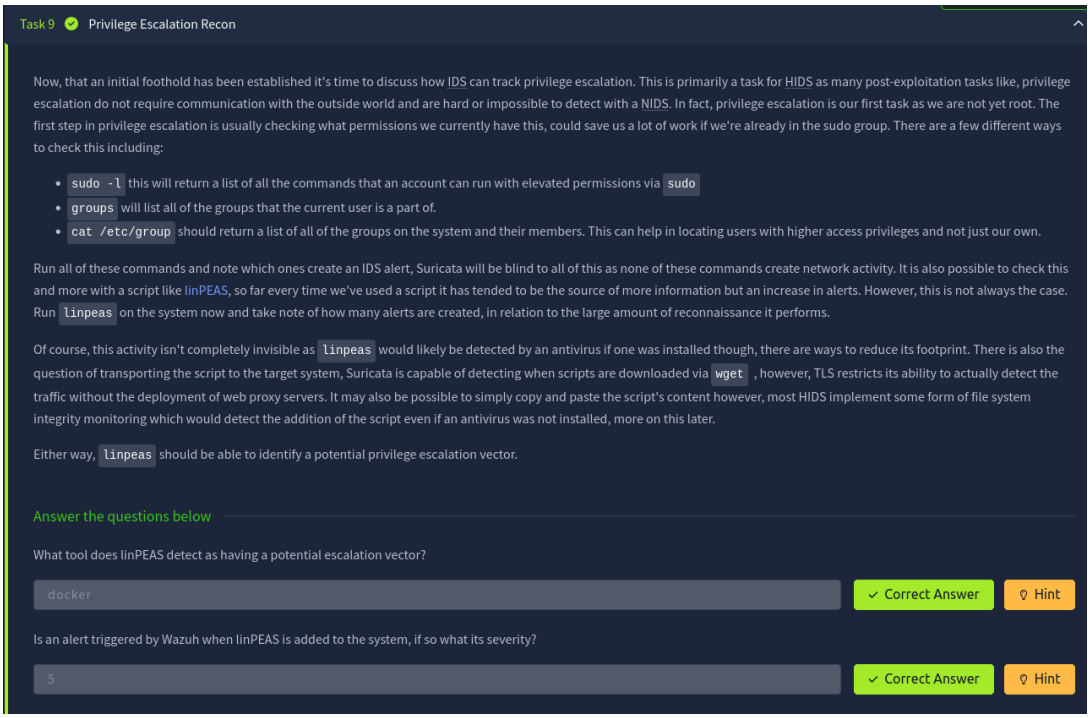

Figure 14: `linpeas.sh` output highlighting the `docker` group.

10 Task 10: Performing Privilege Escalation

Objective

Exploit the `docker` group misconfiguration to gain root privileges.

Methodology

I ran a `docker` command to mount the host's root filesystem ('/') to `/mnt` inside a new container. This gave me root access to the entire host, allowing me to read the flag from `/mnt/root/root.txt`.

Screenshots

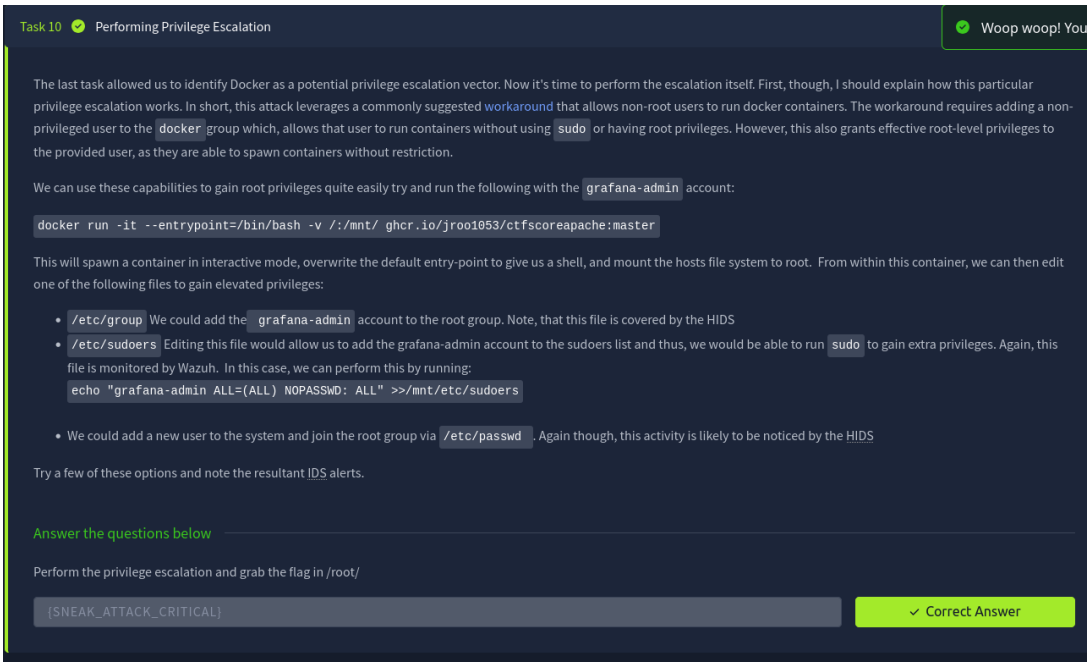

Figure 15: Reading the root flag from `/mnt/root/root.txt` from within the container.

11 Task 11: Establishing Persistence

Objective

Establish a persistent foothold and observe the HIDS detection.

Methodology

From the root container, I added the `grafana-admin` user to the `/mnt/etc/sudoers` file. This file modification was immediately detected by Wazuh, which generated a high-severity alert.

12 Task 12: Conclusion

Summary

This room demonstrated the critical differences and complementary nature of Network IDS (Suricata) and Host IDS (Wazuh).

Key Takeaways

- **NIDS (Suricata)** is strong against known network signatures but blind to encrypted traffic and on-host activity.
- **HIDS (Wazuh)** is strong at detecting file integrity changes, log anomalies, and post-exploitation behavior, but is blind to network-level scans.
- A ”defense-in-depth” strategy, using both, is essential for a robust security posture.

Proof of Completion

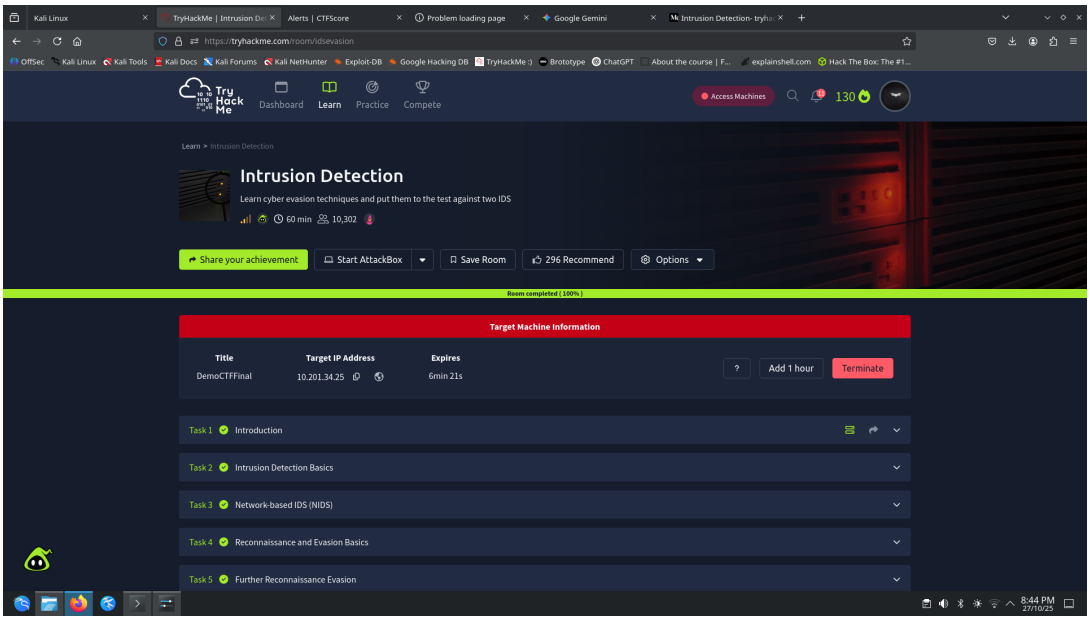

Figure 16: TryHackMe ”IDS Evasion” room completion.